

Sorting Problem 정렬 문제

문제 정의 : 일련의 원소를 특정 순서로 정렬하는 것(e.g. 오름차-nondecreasing, 내림차)

입력 : n개의 일련의 숫자(sequence of n numbers) $a_1, a_2, a_3, \dots, a_n$

출력 : $\langle a_1, a_2, a_3, \dots, a_n \rangle$ 을 permutation(치환, 재정렬, reordering)해서 $a'_1, a'_2, a'_3, \dots, a'_n$ 로 정렬된 값

Sorting이 중요한 이유는? -> 많은 문제들이 쉬워짐!

- * 리스트에서 특정 값을 찾을
- * 최소/최대 값을 찾거나 k번째로 작거나 큰 값을 리스트에서 찾을
- * 중복된 항목을 제거하기 위해서 특정 item의 uniqueness확인
- * 특정 값이 몇 번 나타나는지 세기
- * 금융 거래나 일자, 양 등의 상태를 정렬...

Sorting은 CS에서 기초적인 작업이다. 많은 프로그램이 복잡한 문제를 풀기 위한 중간 단계로 Sorting을 사용.

Sorting의 종류

비교기반 알고리즘 : Bubble S, Insertion S, Selection S, Merge S, Quick S, Heap S...

Non-비교기반 알고리즘 : Counting S, Radix S,...

어떤 알고리즘이 특정 애플리케이션에 맞을까? - Element-Related Factors + Element-Unrelated Factors...

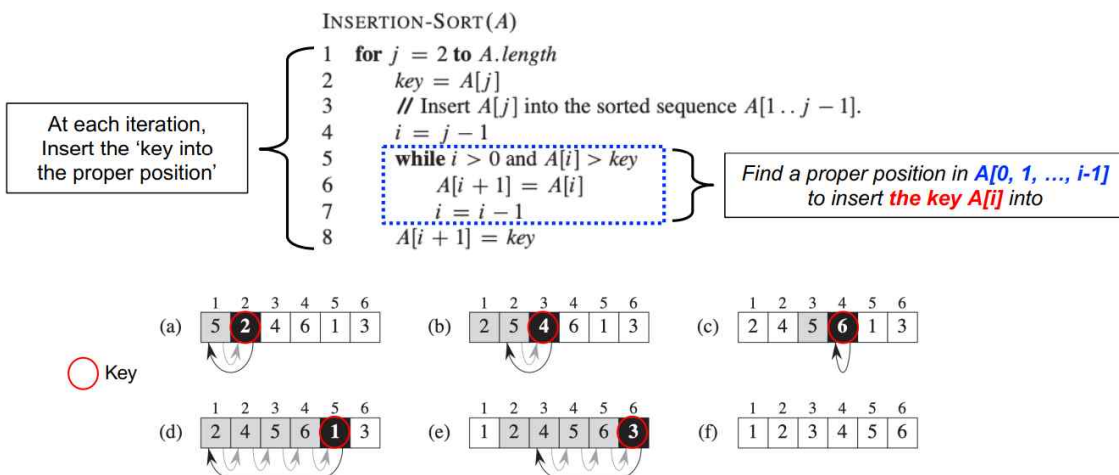
Element-Related Factors : 요소들이 얼마나 이미 정렬되어 있는지, 요소 값에 대한 제약이 있는지

Element-Unrelated Factors : 컴퓨팅 환경(CPU, GPU...) + 스토리지 환경(HDD, SSD...)

Insertion Sort 삽입 정렬

리스트의 정렬되지 않은 부분에서 'Key' 요소를 반복적으로 정렬된 부분으로 계속해서 삽입하는 것.

A sorting algorithm repeatedly inserting a 'key' element from the unsorted portion of the list into the sorted portion of the list while preserving the sorted order.



삽입 정렬은 정확한가(Correct?) : $A[0, 1, \dots, i-1]$ 에 있는 요소들은 정확한 위치에 있다고 보장할 수 있음.

	7	4	3	6	8	1	2
step=1	4	7	3	6	8	1	2
step=2	3	4	7	6	8	1	2
step=3	3	4	6	7	8	1	2
step=4	3	4	6	7	8	1	2
step=5	1	3	4	6	7	8	2
step=6	1	2	3	4	6	7	8

Algorithm 1 Non-increasing Insertion-Sort(A)

```

1: for j = 2 to A.length do
2:   key = A[j]
3:   // Insert A[j] into the sorted sequence A[1..j-1]
4:   i = j - 1
5:   while i > 0 and A[i] < key do
6:     A[i + 1] = A[i]
7:     i = i - 1
8:   end while
9:   A[i + 1] = key
10: end for
    
```

알고리즘의 성능 측정(Performance Analysis of Algorithms)

알고리즘의 측정 기준 : *정확한 결과를 출력하는가? *명확한 instructions을 제공하는가? *효율적으로 작동하나?

속도 측정; Measurement(Machine-dependent): 알고리즘의 실행 시간을 측정. 기계에 따라 측정이 달라짐.

속도 분석; Analysis(Machine-independent): 명령어(inst)의 복잡도를 분석. 시간복잡도, 공간복잡도

알고리즘의 시간복잡도

알고리즘의 시간 복잡도(작동 시간): **primitive instructions의 개수.**

N개의 데이터 원소가 주어지면 몇 개의 inst가 필요한가?

Primitive Instructions의 예

Arithmetic instruction(산술 연산) : + - * / % ceil()-올림 ...

Data movement instructions(데이터 이동) : Load, Store, Copy...

Control instructions(제어문) : Conditional branch(if-else), Unconditional branch(break, continue...),

Subroutine 호출, 리턴

>> 각 명령어는 서로 다른 시간이 걸림. 시간 복잡도를 n의 함수로 표현하려 함.

```

int sum(int list[], int n){
    int i;
    int temp_sum = 0; // assignment
    for (i=0; i<n; i++) // for loop from 0 to n
        temp_sum += list[i]; // add & assignment in for loop
    return temp_sum; // return
}

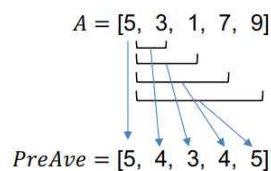
```

Time Complexity of sum():			
Instruction	Cost	Times	
data assign	c_1	1	
for loop (compare & add)	c_2	$n + 1$	
add & assign	c_3	n	
return	c_4	1	
Total = $(c_2 + c_3)n + c_1 + c_2 + c_4$			

c_2 가 $n+1$ 인 이유는 for loop을 종료하기 위해서 조건이 거짓인 것을 비교하는 연산까지 포함해서 $n+1$ 이 됨.

Exercise 1 : Prefix Average(누적 평균) 구하기

$$PreAve[j] = \frac{\sum_{i=0}^j A[i]}{j+1}$$



		Cost	Times
<pre>void prefixAverage(int A[], int PreAve[], int n){ for (int j=0; j<n; j++){ // c_1 int sum = 0; // c_2 for (int i=0; i<j+1; i++) // c_3 sum += A[i]; // c_4 PreAve[j] = sum / (j+1); // c_5 } }</pre>		c_1	
		c_2	
		c_3	
		c_4	
		c_5	

(1) Calculate **the exact number of operations** at each line

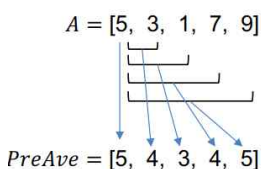
(2) Answer the question "Is this algorithm a linear function of n ?"

$$c_1 : n+1 \quad c_2 : n \quad c_3 : \frac{n(n+3)}{2} = \frac{n((1+1) + (n+1))}{2} \quad c_4 : \frac{n(n+1)}{2} \quad c_5 : n$$

-> Time complexity == $T(n^2)$

Exercise 2: Linear time을 Prefix Averages 구하기

$$PreAve[j] = \frac{\sum_{i=0}^j A[i]}{j+1}$$



		Cost	Times
<pre>void prefixAverageLinear(int A[], int PreAve[], int n){ int sum = 0; // c_1 for (int j = 0; j < n; j++) { // c_2 sum += A[j]; // c_3 PreAve[j] = sum / (j + 1); // c_4 } } // Only one single loop is needed</pre>		c_1	1
		c_2	$n+1$
		c_3	n
		c_4	n

(1) Implement an algorithm that solves this problem in linear time

(2) Calculate the number of operations at each line

Time Complexity of Various Types of Instructions

Loops

```
int m = 0; // constant c_0
for (int i=0; i<n; i++) // constant c_1 * (n+1)
    m = m + 1; // constant c_2 * (n)
```

$$c_0 + c_1(n+1) + c_2n = c_3n + c_4$$

Nested Loop

```
int m = 0; // constant c_0
for (int i=0; i<n; i++) // constant c_1 * (n+1)
    for (int j=0; j<n; j++) // constant c_2 * (n+1) * (n)
        m = m + 1; // constant c_3 * n * n
```

$$c_0 + c_1(n+1) + c_2n(n+1) + c_3n(n+1) = c_4n^2 + c_5n + c_6$$

Consecutive statements

```
int a = 0, b = 0; // constant c_0
a = a + 3; // constant c_1
for (int i=0; i<n; i++) // constant c_2 * (n+1)
    a = a + 2; // constant c_3 * n
for (int i=0; i<n; i++) // constant c_4 * (n+1)
    for (int j=0; j<n; j++) // constant c_5 * (n+1) * (n)
        b = b + 2; // constant c_6 * (n) * (n)
```

$$c_0 + c_1n + c_2n^2$$

If-then-else statements

```
int m = 0; // constant c_0
if (n==0){ // constant c_1
    return false; // constant c_2
}
else{
    for (int i=0; i<n; i++) // constant c_3 * n+1
        m = m + i // constant c_4 * n
}
```

$$c_0 + c_3(n+1) + c_4n$$

Insertion Sort(삽입정렬)의 시간복잡도

INSERTION-SORT(A)

```
1 for j = 2 to A.length
2     key = A[j]
3     // Insert A[j] into the sorted
      sequence A[1..j-1].
4     i = j - 1
5     while i > 0 and A[i] > key
6         A[i+1] = A[i]
7         i = i - 1
8     A[i+1] = key
```

cost times

c_1	n
c_2	$n-1$
0	$n-1$
c_4	$n-1$
c_5	$\sum_{j=2}^n t_j$
c_6	$\sum_{j=2}^n (t_j - 1)$
c_7	$\sum_{j=2}^n (t_j - 1)$
c_8	$n-1$

파란 박스 안은 상황에 따라서 달라짐.

: j에 대해 while 루프에서 실행된 횟수

얼마나 이미 정렬되어있는가에 따라서 t_j 가 달라질 수 있다.

삽입 정렬의 best case - 이미 정렬이 다 되어 있는 경우 ----->

6번줄 7번줄을 실행할 필요가 없어짐!

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

$$\Rightarrow T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

'Linear' function of n

Best Case인 경우는 $O(n)$ 이다.

삽입 정렬의 worst case - 반대로 정렬이 되어 있는 경우

The worst-case running time of Insertion Sort:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

$$\Rightarrow T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1)$$

$$= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n - (c_2 + c_4 + c_5 + c_8)$$

'Quadratic' function of n

★while문에서 False인 경우에는 c_6 와 c_7 이 실행되지 않으므로 1번을 빼준다!

Worst Case인 경우는 $O(n^2)$ 이다.

시간복잡도 분석의 3가지 경우

Best-Case 분석 : 알고리즘이 best-case일 때의 분석. 이미 정렬이 다 되어있는 경우. 의미가 별로 없음.

Average-Case 분석 : 모든 가능한 입력값에 대한 분석. 입력에 따라 달라짐. 분석하기 어려움.

Worst-Case 분석 : 알고리즘이 worst-case일 때의 분석. 반대로 정렬이 되어 있는 경우. **제일 유용함**
>>알고리즘의 Worst-case 작동 시간을 구하는 것이 우리의 목표.

Worst-Case 분석의 중요성

1. 어떠한 입력에 대하여 작동 시간의 상한(upper bound)을 알 수 있음.
이보다 더 늦어지지는 않음.
2. 실생활에서 worst-case가 일어날 수 있음.
3. Average-case가 보통 worst-case 만큼 좋지 않다.

Insertion Sort에서의 공간복잡도(Space Complexity)

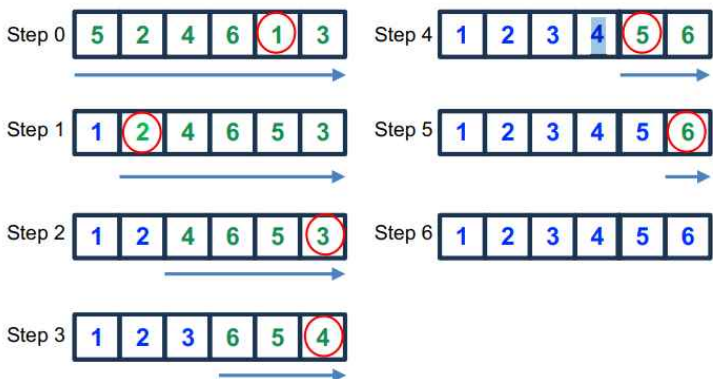
삽입 정렬은 리스트 안에서 값을 이동하기 때문에 **추가적인 공간이 필요하지 않다**.
선언한 상수들에 대해서만 공간이 필요함.

공간복잡도 : $n+c$

> 'In-Place' Sorting Algorithm이다.

Selection Sort(선택 정렬)의 복잡도 분석

선택정렬 : 전체를 스캔해서 최소 값을 찾고 첫 번째 index와 Swap. ...반복...



Algorithm 4 Selection Sort

```
1: for  $i = 1$  to  $n - 1$  do
2:    $min = i$ 
3:   for  $j = i + 1$  to  $n$  do
4:     // Find the index of the  $i^{th}$  smallest element
5:     if  $A[j] < A[min]$  then
6:        $min = j$ 
7:     end if
8:   end for
9:   Swap  $A[min]$  and  $A[i]$ 
10: end for
```

시간복잡도 : Worst-Case와 Best-Case 모두 n^2 ! Quadratic이다.

공간복잡도 : In-Place Sorting이다. Swap을 하기 때문에 추가적인 공간 필요X $n+c$

Asymptotic Analysis(점근적 분석)

복잡도 분석의 역할

- 알고리즘이 얼마나 효율적인지 간단하게 알 수 있음
- 다른 알고리즘 간의 상대적인 성능을 비교할 수 있게 함.

점근적 분석이 중요한 이유

- 정확한 복잡도를 계산하는 것은 때때로 중요함
- 하지만, 정확하게 계산하는 것은 그렇게 가치가 있지 않음.

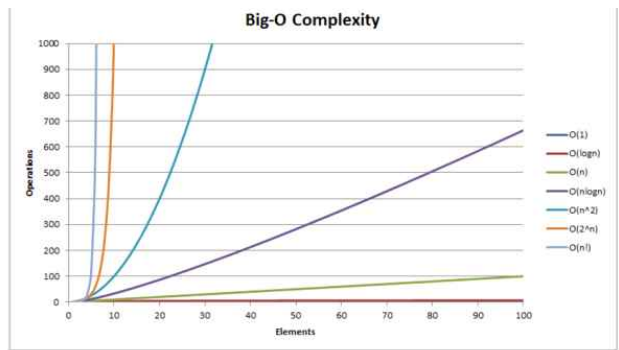
알고리즘에서는 증가하는 정도만 신경쓰!!

input의 크기가 무한대로 커지면 어떻게 될까? -> **알고리즘의 점근적인(Asymptotic) 효율성**

Leading term(최고차항)만 중요하다!

The number of operations

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3	8	24	64	512	256
16	4	16	64	256	4,096	65,536
32	5	32	160	1,024	32,768	4,294,967,296
64	6	64	384	4,096	262,144	1.84×10^{19}
128	7	128	896	16,384	2,097,152	3.40×10^{38}
256	8	256	2,048	65,536	16,777,216	1.15×10^{77}
512	9	512	4,608	262,144	134,217,728	1.34×10^{154}



점근적인 분석이란?

input의 크기가 증가함에 따라 알고리즘의 증가도(growth rate)를 평가하는 것.

왜 쓰는가?

알고리즘이 어떻게 동작하는지 높은 이해를 할 수 있음

특정한 문제에 대해서 알고리즘 간의 효율성을 비교하는데 유용함

큰 input 크기에 대해서 알고리즘이 작동할지 예측할 수 있음

□ O -Notation: "Less than or equal to"

■ $f(n) = O(g(n)) \approx f(n) \leq g(n)$

□ o -Notation: "Less than"

■ $f(n) = o(g(n)) \approx f(n) < g(n)$

□ Ω -Notation: "Greater than or Equal to"

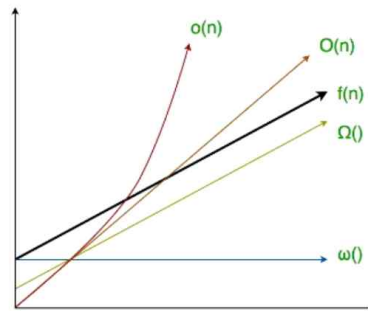
■ $f(n) = \Omega(g(n)) \approx f(n) \geq g(n)$

□ ω -Notation: "Greater than"

■ $f(n) = \omega(g(n)) \approx f(n) > g(n)$

□ Θ -Notation: "Equal to"

■ $f(n) = \Theta(g(n)) \approx f(n) = g(n)$



O-Notation 빅오 표기법

$O(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}.$

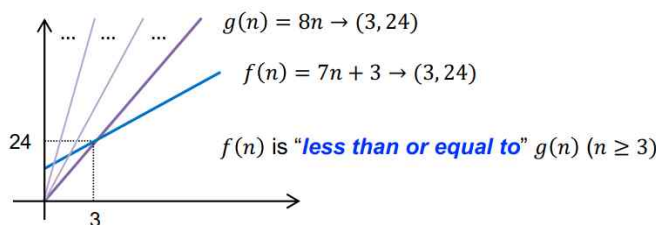
★ 보다 오른쪽에 있는 모든 n 에 대하여 $f(n)$ 은 $c \cdot g(n)$ 보다 작거나 같다

★ $g(n)$ 은 $f(n)$ 의 점근적 상한 asymptotic upper bound이다

예시 1

$f(n) = 7n + 3 = O(n)$ 를 보여라.

□ $c = 8$ and $n_0 = 3, c = 10$ and $n_0 = 1, c = 100$ and $n_0 = 1, \dots$



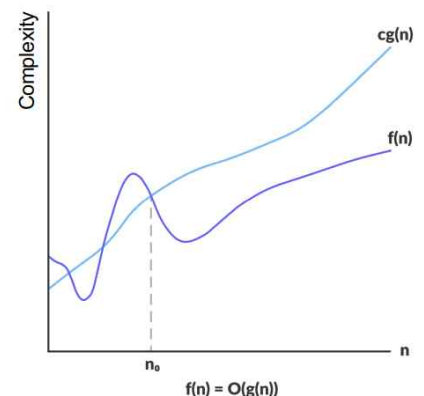
■ There exist a positive constant c and n_0 , such that $3n^3 + 7n + 5 \leq cn$ for $n > n_0$

예시 2

$f(n) = 3n^3 + 7n + 5 = O(n^3)$ 임을 보여라

For all $n > n_0 = 1, 3n^3 + 7n + 5 \leq (3 + 7 + 5)n^3$

$\leq 15n^3 \rightarrow c = 15$ and $n_0 = 1$



예시 3

$$(n) = 3n + 1 = (n) \text{ 을 보여라 } 3n + 1 \leq cn^2 \text{ for all } n > n_0$$

$$\text{Dividing by } n^2 \longrightarrow \frac{3}{n} + \frac{1}{n^2} \leq c$$

The inequality holds for any $n \geq 1$ and $c \geq 4$

→ Thus, $f(n) = O(n^2)$
Similarly, $f(n) = O(n^3)$, $f(n) = O(n^4)$, ...

Is the asymptotic upper bound provided by O-notation **asymptotically tight**?

>> 너무 느슨하다..

o-Notation 스몰 오 표기법 - 점근적으로 tight하지 않은 상한

$o(g(n)) = \{f(n): \text{for any positive constants } c > 0, \text{ there exist a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0\}.$

모든 n_0 보다 오른쪽에 있는 n 에 대해서 $f(n)$ 은 $c \cdot g(n)$ 보다 훨씬 작다. 절대 같을 수 없다.
 $f(n)$ 은 $g(n)$ 보다 점근적으로 작다.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

O-Notation vs o-notation

$f(n) = O(g(n))$ 0 $f(n) \leq cg(n)$ some 몇몇 양의 정수에 대하여 성립함

$f(n) = o(g(n))$ 0 $f(n) < cg(n)$ all 모든 양의 정수에 대하여 성립함

For $f(n) = 2n + 3$, we have:

- $f(n) = O(n)$ but $f(n) \neq o(n)$ Asymptotically tight
- $f(n) = O(n^2)$ and $f(n) = o(n^2)$
- $f(n) = O(n^3)$ and $f(n) = o(n^3)$

Ω -Notation 빅 오메가 표기법

$\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n > n_0\}.$

★ n_0 보다 오른쪽에 있는 모든 n 에 대하여 $f(n)$ 은 $c \cdot g(n)$ 보다 작거나 같다

★ $g(n)$ 은 $f(n)$ 의 점근적 상한 asymptotic upper bound이다3

예시 1 $f(n) = 3n^2 + 4n + 1 = \Omega(n)$ 을 보여라

□ Example 1: Show that $f(n) = 3n^2 + 4n + 1 = \Omega(n)$

$$3n^2 + 4n + 1 \geq cn \text{ for all } n > n_0$$

$$\text{Dividing by } n \longrightarrow 3n + 4 + \frac{1}{n} \geq c$$

The inequality holds for any $n \geq 2$ and $c \geq 2$

→ Thus, $f(n) = \Omega(n)$

Is the asymptotic lower bound provided by Ω -notation tight enough?

>> 더 tight 한 것은 $\Omega(n^2)$

ω-Notation 스몰 오메가 표기법

점근적으로 tight 하지 않은 하한

$\omega(g(n)) = \{f(n) : \text{for any positive constants } c > 0, \text{ there exist a constant } n_0 > 0 \text{ such that } 0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0\}.$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

모든 c 보다 오른쪽에 있는 n 에 대해서 $f(n)$ 은 $c \cdot g(n)$ 보다 훨씬 크다. 절대 같을 수 없다.
 $f(n)$ 은 $g(n)$ 보다 점근적으로 크다.

Ω-Notation vs ω-notation

$f(n) = \Omega(g(n)) \iff 0 < cg(n) \leq f(n)$ some 몇몇 양의 정수에 대하여 성립함

$f(n) = \omega(g(n)) \iff 0 < cg(n) < f(n)$ all 모든 양의 정수에 대하여 성립함

For $f(n) = 2n^3 + 3$, we have:

• $f(n) = \Omega(n)$ and $f(n) = \omega(n)$

• $f(n) = \Omega(n^2)$ and $f(n) = \omega(n^2)$

• $f(n) = \Omega(n^3)$ but $f(n) \neq \omega(n^3)$

Asymptotically tight

Q. 그런데... Ω 와 ω 만으로 충분할까?

A. 의미가 없을 수도 있음.

Θ-Notation 세타 표기법

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n > n_0\}.$

> $f(n)$ 은 $g(n)$ 과 같다!

> $g(n)$ 은 $f(n)$ 의 점근적 tight bound이다.

□ **Example 1:** Show that $f(n) = \frac{1}{2}n^2 - 3n = \Theta(n^2)$

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2 \text{ for all } n > n_0$$

Dividing by n^2

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

- The left-hand side of the inequality holds for any $n \geq 7$ and $c_1 \geq \frac{1}{14}$
- The right-hand side of the inequality holds for any $n \geq 1$ and $c_2 \geq \frac{1}{2}$

Thus, the inequality holds when $c_1 \geq \frac{1}{14}$, $c_2 \geq \frac{1}{2}$, and $n_0 \geq 7$

정리(Theorem) :

For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$
 if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$

Upper bound

Lower bound

Tight bound

$f(n) = \Theta(g(n))$ 이라면, $f(n) = O(g(n))$ 과 $f(n) = \Omega(g(n))$ 을 만족시킨다.

여기서 알 수 있는 것

□ Lesson

For any polynomial $p(n) = \sum_{i=0}^d a_i n^i$ where the a_i are coefficients and $a_d > 0$, we have $p(n) = \Theta(n^d)$

□ a_d : the coefficient of the highest-order term

■ Example: quadratic function $f(n) = an^2 + bn + c$ where $a > 0$, $f(n) = \Theta(n^2)$

□ Throwing away the lower-order terms and ignoring the coefficient

>> 최고차항이 k인 다항식이 있다면, $\Theta(n^k)$ 이다

점근적 표기의 특성

비교 기능(Comparing Function)

두 실수를 비교하는 것처럼 둘 함수를 쉽게 비교할 수 있다

Transitivity(전이성) : 의미가 연쇄적으로 유지됨.

■ $f(n) = O(g(n))$ and $g(n) = O(h(n)) \rightarrow f(n) = O(h(n))$

□ $\approx f(n) \leq g(n)$ and $g(n) \leq h(n) \rightarrow f(n) \leq h(n)$

■ $f(n) = o(g(n))$ and $g(n) = o(h(n)) \rightarrow f(n) = o(h(n))$

□ $\approx f(n) < g(n)$ and $g(n) < h(n) \rightarrow f(n) < h(n)$

■ $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n)) \rightarrow f(n) = \Omega(h(n))$

□ $\approx f(n) \geq g(n)$ and $g(n) \geq h(n) \rightarrow f(n) \geq h(n)$

■ $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n)) \rightarrow f(n) = \omega(h(n))$

□ $\approx f(n) > g(n)$ and $g(n) > h(n) \rightarrow f(n) > h(n)$

■ $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \rightarrow f(n) = \Theta(h(n))$

□ $\approx f(n) = g(n)$ and $g(n) = h(n) \rightarrow f(n) = h(n)$

Reflexivity (반사성) : 자기 자신과의 관계가 성립함

■ $f(n) = O(f(n))$

□ $\approx f(n) \leq f(n)$

e.g., $f(n) = n^3 = O(n^3)$

■ $f(n) = \Omega(f(n))$

□ $\approx f(n) \geq f(n)$

e.g., $f(n) = an^2 + bn + c = \Omega(an^2 + bn + c) = \Omega(n^2)$

■ $f(n) = \Theta(f(n))$

□ $\approx f(n) = f(n)$

e.g., $f(n) = n^2 + bn = \Theta(n^2)$

$\rightarrow f(n)$ is an asymptotical tight bound of $f(n)$

(i.e., there exist constants c and n_0 satisfying the conditions of notation)

Symmetry(대칭성)

□ Symmetry of Asymptotic Notations (=)

■ $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$

□ $\approx f(n) = g(n)$

Transpose Symmetry (전치 대칭성)

- $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$
 $\square \approx f(n) \leq g(n) \leftrightarrow g(n) \geq f(n)$
- $f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$
 $\square \approx f(n) < g(n) \leftrightarrow g(n) > f(n)$

Trichotomy(삼분법) - 실수에서는 성립!

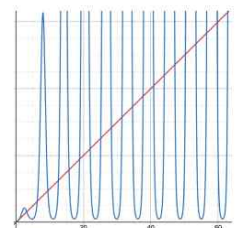
어떤 실수 a, b 가 있으면 이 세 관계 중 하나에 무조건 성립해야 한다.

$$a \leq b, a = b, a \geq b$$

그럼 두 함수 $f(n), g(n)$ 에서도 성립할까(점근적으로 비교가능한가; Asymptotically comparable)? => **NO!**
 $f(n)$ 이 $O(g(n))$ 도 아니고 $(g(n))$ 도 아닌 $f(n)$ 이 있을까? => **YES!**

점근적 비교에서는 삼분법이 성립하지 않는다!

반례 : ----->



$$f(n) = n \text{ vs } g(n) = n^{1+\sin n}$$

Asymptotic Notations : Exercice

Exercise 1 : $2^{n+1} = (2^n)?$

양변을 2^n 으로 나누면 $2 \leq c$ 이므로 모든 n 에 대하여 $2 \leq c$ 이다.

-> 참이다!

Exercise 2 : $2^{2n} = O(2^n)?$

양변을 2^n 으로 나누면 $2^n \leq c$ 가 되는데 어떤 n 을 넣더라도 어떠한 상수 c 보다는 커지기 때문에 이 부등식은 참이 될 수 없다. -> 거짓이다!

Exercise 3 :

$\max(f(n), g(n)) = \Theta(f(n) + g(n))$ 을 1) Θ 표기법의 정의와 2) Θ 표기법의 정리(theorem)으로 보여라.

1) Θ 표기법의 정의(definition) $f(n) \geq 0, g(n) \geq 0$

There exist constants c_1, c_2 , and n_0 such that
 $c_1(f(n) + g(n)) \leq \max(f(n), g(n)) \leq c_2(f(n) + g(n))$ for all $n > n_0$.

$$\frac{1}{2} * (f(n) + g(n)) \leq \max(f(n), g(n)) \leq 1 * (f(n) + g(n))$$

$$\because f(n) \geq 0 \text{ and } g(n) \geq 0$$

$$\text{e.g., } \frac{1}{2} * (6+4) \leq \max(6,4) \leq (6+4)$$

This inequality holds for any $c_1 \leq \frac{1}{2}$ and $c_2 \geq 1$

$$\therefore \max(f(n), g(n)) = \Theta(f(n) + g(n))$$

By the definition of Θ -notation

2) 표기법의 정리(theorem)

□ **Exercise 3:** show $\max(f(n), g(n)) = \Theta(f(n) + g(n))$

■ Upper bound of $\max(f(n), g(n))$

$$f(n) \geq 0 \text{ and } g(n) \geq 0 \longrightarrow \begin{matrix} f(n) \leq f(n) + g(n) \\ g(n) \leq f(n) + g(n) \end{matrix}$$

$$\therefore \max(f(n), g(n)) = O(f(n) + g(n))$$

■ Lower bound $\max(f(n), g(n))$

$$\begin{matrix} \text{If } f(n) \geq g(n) \rightarrow f(n) + g(n) \leq 2 * f(n) \\ \text{If } f(n) \leq g(n) \rightarrow f(n) + g(n) \leq 2 * g(n) \end{matrix} \xrightarrow{\text{Dividing by 2}} \frac{1}{2} * (f(n) + g(n)) \leq \max(f(n), g(n))$$

$$\therefore \max(f(n), g(n)) = \Omega(f(n) + g(n))$$

By the theorem of Θ -notation

▲ **Upper Bound (Big-O) 증명**

전제: $f(n) \geq 0, g(n) \geq 0$ 라고 했을 때,

- $f(n) \leq f(n) + g(n)$
 - $g(n) \leq f(n) + g(n)$
- ☞ 그러므로 $\max(f(n), g(n)) \leq f(n) + g(n)$

이건 바로:

$$\max(f(n), g(n)) = O(f(n) + g(n))$$

즉, 최대값은 $f(n) + g(n)$ 보다 더 커질 수는 없다.

▼ **Lower Bound (Big-Omega) 증명**

이번엔 반대로 **최대값이 얼마나 작아질 수 있는가**를 봅니다.

- 만약 $f(n) \geq g(n)$ 이면
 $f(n) + g(n) \leq 2 * f(n)$
 $\rightarrow$ 즉, $f(n) \geq (f(n) + g(n)) / 2$
- 만약 $f(n) \leq g(n)$ 이면
 $f(n) + g(n) \leq 2 * g(n)$
 $\rightarrow$ 즉, $g(n) \geq (f(n) + g(n)) / 2$

두 경우 다 결국:

$$\max(f(n), g(n)) \geq \frac{1}{2} * (f(n) + g(n))$$

☞ 즉,

$$\max(f(n), g(n)) = \Omega(f(n) + g(n))$$

Exercise 4 :

F a. $f(n) = O(g(n))$ implies $g(n) = O(f(n))$.

$f(n) = n, g(n) = n$ 이면 성립하지 않음

F b. $f(n) + g(n) = \Theta(\min(f(n), g(n)))$.

$f(n) = n, g(n) = n^2$ 이라면 성립하지 않음.

T c. $f(n) = O(g(n))$ implies $\lg(f(n)) = O(\lg(g(n)))$, where $\lg(g(n)) \geq 1$ and $f(n) \geq 1$ for all sufficiently large n .

양변에 로그를 취하더라도 증가속도는 같음! 따라서 참.

F d. $f(n) = O(g(n))$ implies $2^{f(n)} = O(2^{g(n)})$.

$f(n) = 2n, g(n) = n$ 이면 $2^{2n} \neq O(2^n)$ 이 성립하지 않으므로 거짓.

F e. $f(n) = O((f(n))^2)$.

$f(n) = \frac{1}{n}$ 이면 $\frac{1}{n} \neq O(\frac{1}{n^2})$ 이므로 성립하지 않으므로 거짓.

T f. $f(n) = O(g(n))$ implies $g(n) = \Omega(f(n))$.

a b, b a 정의에 따라 참.

F g. $f(n) = \Theta(f(n/2))$.

$f(n) = 2^n$ 이면 성립하지 않음. $2^{2n} \neq O(2^n)$ 이 성립하지 않음. 거짓.

T h. $f(n) + o(f(n)) = \Theta(f(n))$.

스몰 o는 f(n)에 비하면 차수가 한참 낮음. 결과에 영향X
따라서 결과는 $f(n) = \Theta(f(n))$ 과 같으므로 참

3주차 Divide and conquer

Divide and Conquer Approach 분할정복 접근

1. 큰 문제를 작은 서브문제로 쪼갬다. Break, Divide
2. 각각의 서브문제를 재귀적으로(recursively) 해결한다. Solve, Conquer
3. 각각의 문제들을 원래 문제로 합친다. Combining

적용

Merge Sort(병합정렬), Quick Sort(퀵정렬)

Max, Min(최대, 최소) 찾기

특징 - 왜 분할정복이 중요할까?

Simplicity 단순성 : 쉽고 간단한 이해와 구현

Efficiency 효율성 : 시간복잡도가 좋음. 특히 $n \log n$

Parallelism 병렬성 : 병렬화되기 좋음. 빠름

고려할 점

높은 메모리 사용 High memory usage : 많은 메모리 공간이 필요할 수 있음

낮은 적용성 Low applicability : 몇몇 특정한 문제들에게만 적절함.

분석의 어려움 Difficulty of analysis :

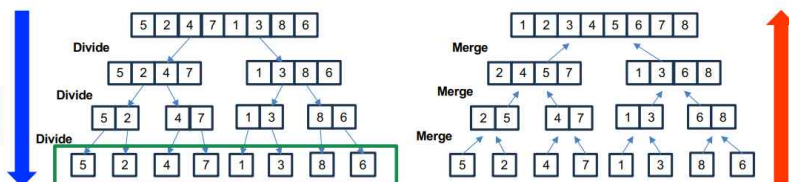
Merge Sort 병합정렬

1. Divide array를 두 개의 subarray로 나눔.
2. Conquer 각각의 반절 크기 array를 재귀적으로 Merge sort(병합정렬)함.
만약 사이즈가 1이라면 할 필요 없음.
3. Combine 쪼갬 subarray를 합쳐서 최종 array로 만듦.

MERGE-SORT(A, p, r)

```

1  if  $p < r$ 
2     $q = \lfloor (p+r)/2 \rfloor$ 
3    MERGE-SORT( $A, p, q$ )
4    MERGE-SORT( $A, q+1, r$ )
5    MERGE( $A, p, q, r$ )
    
```



If a subproblem is small enough (i.e., the array size is 1),
We can solve them in a straightforward manner

How merge two subarrays?

변수설명 A: original input p: 처음 index q: 중간 인덱스 r : 마지막 인덱스

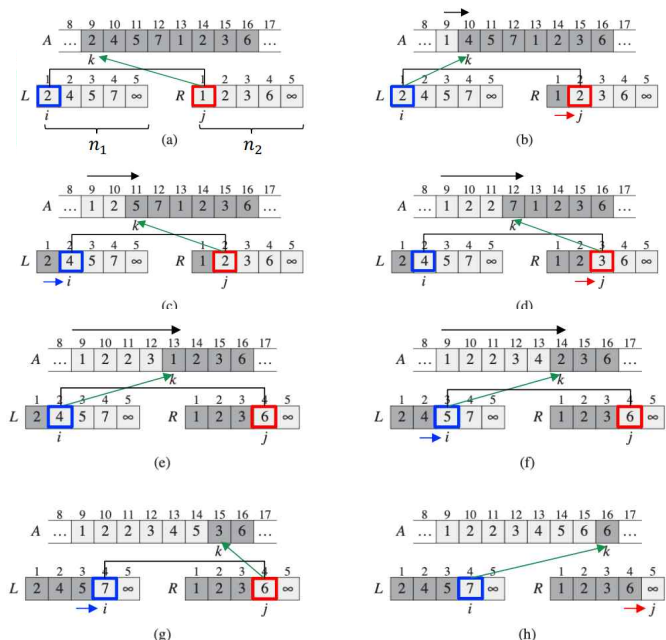
MERGE(A, p, q, r)

```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1+1]$  and  $R[1..n_2+1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5     $L[i] = A[p+i-1]$ 
6  for  $j = 1$  to  $n_2$ 
7     $R[j] = A[q+j]$ 
8   $L[n_1+1] = \infty$ 
9   $R[n_2+1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
    
```

Require extra space

Just copy



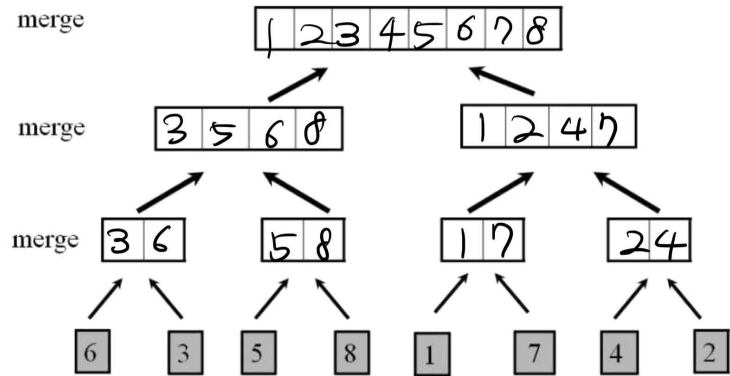
Merge Sort의 정확도

반 크기의 정렬된 subarray를 합쳐서 정렬된 subarray를 얻음 => 정확하다!

연습 1 : Merge Sort를 그려라

Exercise 1: Process of Merge Sort

■ Illustrate the operation of merge sort on the array <6, 3, 5, 8, 1, 7, 4, 2>



연습 2 : sentinels 없이 코드를 재작성해라.

MERGE(A, p, q, r)

```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
    
```

Sentinels

```

for  $k = p$  to  $r$ 
    if  $i > n_1$ 
         $A[k] = R[j]$ 
         $j = j + 1$ 
    else if  $j > n_2$ 
         $A[k] = L[i]$ 
         $i = i + 1$ 
    else if  $L[i] \leq R[j]$ 
         $A[k] = L[i]$ 
         $i = i + 1$ 
    else
         $A[k] = R[j]$ 
         $j = j + 1$ 
    
```

조건문

역할

if $i > n_1$	R의 남은 원소를 복사
else if $j > n_2$	L의 남은 원소를 복사
else if $L[i] \leq R[j]$	L의 현재 값을 복사
else	R의 현재 값을 복사

Merger Sort의 시간복잡도

1. **Divide** $D(n) = (1)$ 단순히 subarray의 중간 인덱스를 계산하는 것
2. **Conquer** $2 * T(\frac{n}{2})$ 재귀적으로 두 subprogram을 해결
3. **Combine** $C(n) = \theta(n)$ $n/2$ 크기의 리스트를 합침

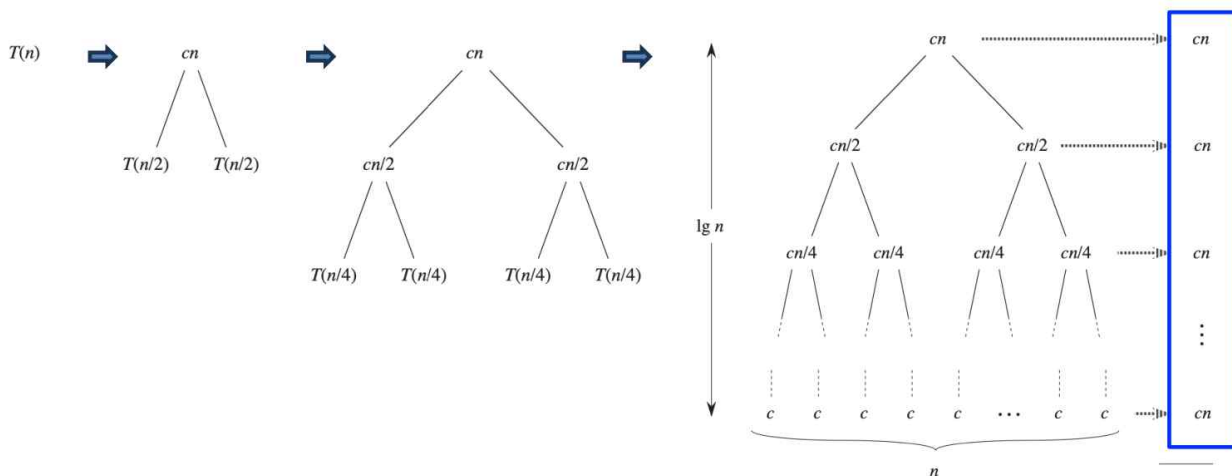
Recursion Tree : 재귀 트리

$(n) = 2 * T(\frac{n}{2}) + \theta(n)$ // $\theta(n)$ 은 현재 단계에서 수행한 입력크기에 비례하는 작업 수(병합 등)

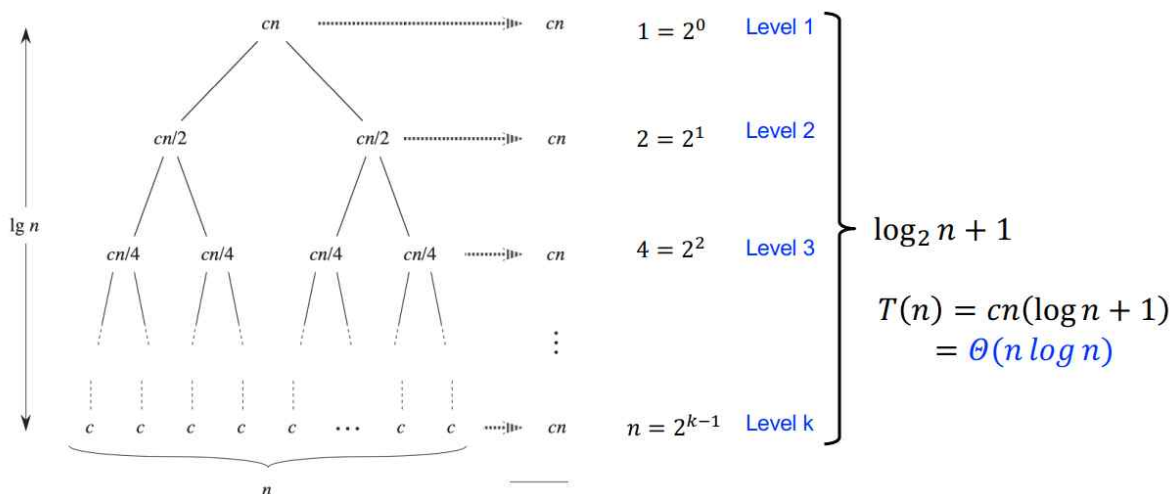
// $\theta(n) = cn$

□ Recursion Tree: Solving the Recurrence

$$T(n) = 2 * T(n/2) + \theta(n) \longrightarrow T(n) = 2 * T(n/2) + cn$$



$$T(n) = 2 * T(n/2) + \theta(n) \longrightarrow T(n) = 2 * T(n/2) + cn$$



$$\therefore T(n) = cn(\log n + 1) = \Theta(n \log n)$$

//1이 붙는 경우는 마지막 leaf node 까지 더하기 때문
//8을 예로 들면 $\log_2 8 + 1 = 3 + 1 = 4$ 단계를 수행함.

Best, Worst case 비교

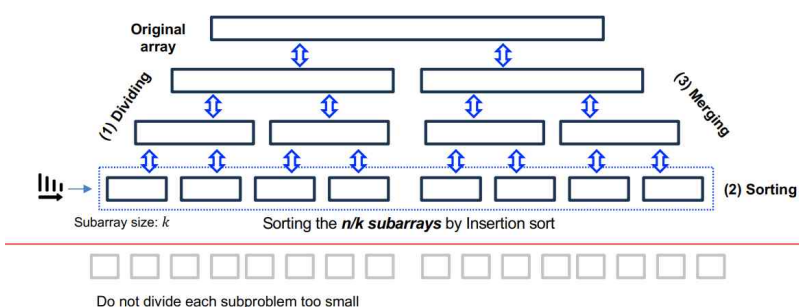
Best - $\Theta(n \log n)$

Worst - $\Theta(n \log n)$ >> Best나 Worst나 시간은 똑같이 걸린다

Q) subarray가 충분히 작을때도 merge sort가 효율적일까..?

Merge Sort 개선하기

초기에 나오는 들은 insertion sort를 하면 더 빨라지지 않을까?



1. Sort insertion sort 로 정렬한다. Best - k Worst - n/k

□ Best case: $\theta\left(k * \left(\frac{n}{k}\right)\right) = \theta(n)$

□ Worst case: $\theta\left(k^2 * \left(\frac{n}{k}\right)\right) = \theta(nk)$

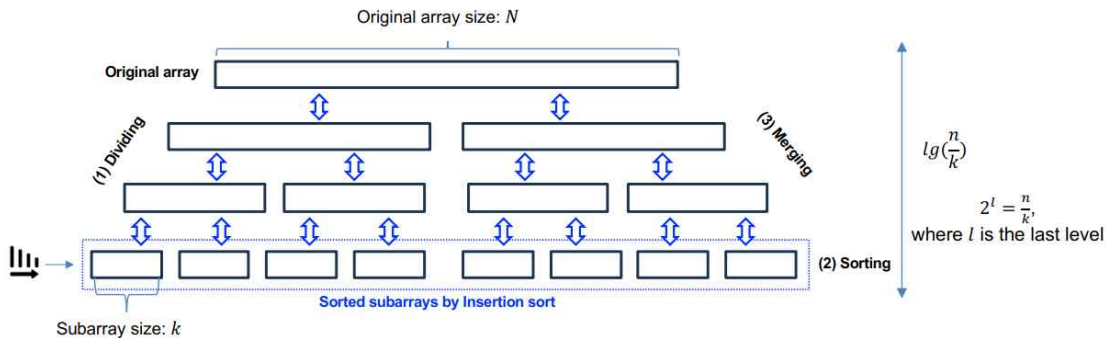
n : the size of the original array
 k : the size of the subarray that insertion sort sorts

2. Merge n/k개의 subarray를 병합한다. worst나 best나 똑같음.

□ Worst case: (# of levels) * (The cost of merging at each level) = $\lg\left(\frac{n}{k}\right) * \theta(n) = \theta\left(n\lg\left(\frac{n}{k}\right)\right)$

■ The total cost of the algorithm (Merge + Insertion) is as follows:

- Best case: $\theta(n) + \theta\left(n\lg\left(\frac{n}{k}\right)\right) = \theta\left(n\lg\left(\frac{n}{k}\right)\right)$ Merge sort: $\theta(n\lg n)$
- Worst case: $\theta(kn) + \theta\left(n\lg\left(\frac{n}{k}\right)\right) = \theta\left(n\lg\left(\frac{n}{k}\right)\right)$ vs Insertion sort: $\theta(n^2)$



>> 조금 더 빨라짐!

Recurrence(점화식)

점화식으로 표현될 수 있다!

점화식 : 어떤 함수를 더 작은 입력값에서의 함수값으로 표현하는 방정식.

점화식을 푸는 것은? 솔루션에서 점근적 나 O를 얻기 위한 것.

점화식을 풀기 위한 세 가지 방법

1. Substitution method 대입법
2. Recursion-tree 재귀트리 그리기
3. Master method 마스터 방법

기술적 사항(알아두기)

인자에는 정수만 들어간다고 가정.

boundary condition을 생략해도 됨.

Boundary conditions are omitted

$$T[n] = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

Only defined when n is an integer

1. Substitution Method 대입법

Step 1 Guess the solution. 솔루션을 추정한다.

Step 2 맞다는 것을 밝히기 위해 수식을 작성한다.

◆ 점화식과 귀납 가정 다시 정리

✓ 점화식:

$$T(n) = 2T(n/2) + n$$

이건 Merge Sort처럼 "크기 n 문제를 두 개의 $n/2$ 문제로 나눈 뒤, 병합에 n 시간이 든다"는 의미입니다.

✓ 귀납 가정 (Inductive Hypothesis)

수학적 귀납법에서 다음을 **가정(Assume)**합니다:

$$T(n/2) \leq c \cdot \frac{n}{2} \log\left(\frac{n}{2}\right)$$

• 즉, n 이 아니라 $n/2$ 일 때는 우리가 원하는 $O(n \log n)$ 형태로 이미 성립한다고 가정하는 겁니다.

• 이걸 점화식에 대입해서 $T(n) \leq cn \log n$ 이 성립하는지를 확인하려는 거예요.

□ Example: $T(n) = 2 * T\left(\frac{n}{2}\right) + n$

■ Guess: $T(n) = O(n \lg n)$

■ Prove: $T(n) \leq cn \lg n$ ($c > 0$)

$$\begin{aligned} T(n) &\leq 2 * c \left(\frac{n}{2} \lg \frac{n}{2}\right) + n \\ &= cn(\lg n - \lg 2) + n \\ &= cn \lg n - cn + n \\ &\leq cn \lg n \quad (\text{as long as } c \geq 1) \end{aligned}$$

Substitution Method

CAU

Two Steps of Substitution Method

1. Guess the solution
 - How to guess a good solution?
2. Use mathematical induction to prove that the guess is right

Example: $T(n) = 2 * T\left(\frac{n}{2}\right) + n$

Guess: $T(n) = O(n \lg n)$

Prove: $T(n) \leq cn \lg n$ ($c > 0$)

$$\begin{aligned}
 T(n) &\leq 2 * c \left(\frac{n}{2}\right) \lg \frac{n}{2} + n \\
 &= cn(\lg n - \lg 2) + n \\
 &= cn \lg n - cn + n \\
 &\leq cn \lg n \quad (\text{as long as } c \geq 1)
 \end{aligned}$$

Handwritten notes: $cn \lg \frac{n}{2} = cn(\lg n - \lg 2)$, $cn \lg n - cn + n$, $cn \lg n - n(c-1)$

31

Substitution Method: Exercises

Exercise 1:

- Show that the solution of $T(n) = T(n-1) + n$ is $O(n^2)$

Prove: $T(n) \leq cn^2$

$$\begin{aligned}
 T(n) &\leq c(n-1)^2 + n \\
 &= cn^2 - 2cn + c + n \\
 &= cn^2 + n(1-2c) + c \\
 &\leq cn^2 \quad (\text{as long as } c > \frac{1}{2})
 \end{aligned}$$

Exercise 2:

- Show that the solution of $T(n) = T(n/2) + 1$ is $O(\lg n)$

Prove: $T(n) \leq c \lg n$

$$\begin{aligned}
 T(n) &\leq c \lg \frac{n}{2} + 1 \\
 &= c \lg n - c \lg 2 + 1 \\
 &\leq c \lg n \quad (\text{as long as } c \geq 1)
 \end{aligned}$$